

Lab 6: Naïve Bayes Classification

Course-	M.Tech.	Type- Core	Core
Course Code-	ABAI5005L	Course Name-	Advanced Machine Learning
Year-	2026-27	Semester-	ODD
Date-	TBD	Batch-	1

Scope of this lab: This lab develops an understanding of the Naïve Bayes classification algorithm, covering prior probability, conditional probability, posterior probability, the conditional independence assumption, and the effect of smoothing and preprocessing on classification performance. Duration: 2 Hours. Dataset: Pima Indians Diabetes Dataset — Kaggle. Model: Gaussian Naïve Bayes. Software: Python 3.x, Jupyter Notebook / Google Colab.

Learning Objectives

After completing this laboratory, students will be able to:

- Understand and apply prior, conditional and posterior probability in a classification setting.
- Implement the Naïve Bayes classification algorithm, including the conditional independence assumption.
- Estimate class-conditional probabilities using the Gaussian probability density function.
- Build and evaluate a Gaussian Naïve Bayes classifier using Scikit-Learn.
- Examine how preprocessing, feature scaling and feature correlation affect Naïve Bayes performance.

Background

Part A: Bayes' Theorem and the Naïve Assumption

Naïve Bayes classifiers apply Bayes' theorem to estimate the posterior probability of each class given a set of features: $P(C|X) \propto P(C) \cdot P(X|C)$. The classifier is called "naïve" because it assumes that, given the class, the features are conditionally independent of one another, which allows the joint likelihood $P(X|C)$ to be approximated as the product of the individual feature likelihoods $P(X_i|C)$.

Part B: Gaussian Naïve Bayes

When the features are continuous, each class-conditional distribution $P(X_i|C)$ can be modelled using a Gaussian (normal) distribution, parameterised by the mean and standard deviation of that feature within each class. The class with the highest posterior probability is chosen as the prediction. Gaussian Naïve Bayes is simple, fast to train, and often performs well even when the independence assumption is only approximately true.

Dataset

Download the Pima Indians Diabetes Dataset from Kaggle.

The dataset contains medical diagnostic measurements such as glucose level, blood pressure, BMI, age, and other attributes, with a binary target indicating diabetes outcome.

Students should:

- Download the dataset from Kaggle.
- Identify the feature and target variables.
- Inspect the data types and class distribution.
- Check for missing/invalid values before modelling.

Dataset	Why it is suitable	Recommended NB
Pima Indians Diabetes Database	Binary classification, mostly numerical features, easy to interpret	GaussianNB
Heart Disease Dataset	Medical classification with mixed numerical/categorical variables; good for discussing feature independence	GaussianNB / CategoricalNB
Breast Cancer Wisconsin Dataset	Binary classification, numerical features, relatively clean	GaussianNB
Wine Quality Dataset	Numerical features and multiclass classification; useful for comparing class probabilities	GaussianNB
Titanic Dataset	Mixed numerical + categorical features; useful for preprocessing discussion	CategoricalNB / GaussianNB after preprocessing

Required Libraries

Library	Purpose
pandas	Loading and exploring the dataset (read_csv, describe, groupby)
numpy	Numerical computations for prior, mean, standard deviation and Gaussian probability calculations
matplotlib.pyplot	Visualization of feature distributions across classes
sklearn.model_selection	Train-test split
sklearn.naive_bayes	Gaussian Naive Bayes classifier
sklearn.preprocessing	Feature scaling (StandardScaler)
sklearn.metrics	Classification evaluation metrics (accuracy, precision, recall, F1, confusion matrix)

Question 1: Dataset Exploration and Probability Analysis

Objective: Load and explore the dataset, and calculate the prior probability of each class.

Instructions:

- Identify the input features and target variable.
- Determine the number of samples and features.
- Examine the distribution of the two target classes.
- Calculate the prior probability ($P(C)$) of each class.
- Identify features containing zero or potentially invalid measurements.
- Select one feature and compare its distribution for the two classes.
- Based on the feature distributions, discuss whether the Gaussian assumption appears reasonable.

Hint — Useful functions/classes:

- `pd.read_csv()`, `df.info()`, `df.describe()`, `df.shape`
- `value_counts()`, `isnull().sum()`
- `groupby()`, `mean()`, `std()`

Question 2: Naïve Bayes Classification from Scratch

Objective: Implement Naïve Bayes classification manually for a small subset of the dataset.

Instructions:

- Calculate the prior probability ($P(C)$) for each class.
- Estimate the mean and standard deviation of a feature for each class.
- Use the Gaussian probability density function to estimate $P(X_i|C)$.
- Calculate the posterior probability: $P(C|X) \propto P(C) \cdot \prod_i P(X_i|C)$.
- Predict the class having the highest posterior probability.
- Repeat the calculation for at least two test samples.
- Explain the role of the conditional independence assumption in the above calculation.

Hint — Useful functions/classes:

- `np.mean()`, `np.std()`, `np.exp()`, `np.sqrt()`
- `np.prod()`, `np.log()`, `np.argmax()`

Question 3: Gaussian Naïve Bayes Using Scikit-Learn

Objective: Build and evaluate a complete Gaussian Naïve Bayes classification model.

Instructions:

- Divide the dataset into training and testing sets.
- Train a Gaussian Naïve Bayes classifier.
- Generate predictions for the test set.
- Evaluate the model using accuracy, precision, recall, F1-score, and the confusion matrix.

- Interpret the false-positive and false-negative predictions.
- Compare the model's performance with the majority-class baseline.
- Explain why GaussianNB is appropriate for this dataset.

Hint — Useful functions/classes:

- `train_test_split()`, `GaussianNB()`, `fit()`, `predict()`
- `predict_proba()`, `accuracy_score()`
- `classification_report()`, `confusion_matrix()`

Question 4: Effect of Preprocessing and Model Assumptions

Objective: Investigate how preprocessing and feature characteristics affect Naïve Bayes.

Instructions:

- Train Gaussian Naïve Bayes using the original features.
- Apply appropriate feature preprocessing/scaling and train the model again.
- Compare the performance of the two models.
- Investigate whether removing highly correlated features changes the performance.
- Compare the predicted probabilities of selected test samples.
- Discuss whether feature scaling significantly affects Gaussian Naïve Bayes.
- Discuss whether feature correlation violates the Naïve Bayes assumption.
- Discuss whether a model with an unrealistic independence assumption can still perform well.
- Identify the major sources of error in this experiment.

Hint — Useful functions/classes:

- `StandardScaler()`, `fit_transform()`, `transform()`
- `corr()`, `np.linalg.norm()`
- `predict_proba()`, `accuracy_score()`

Viva Questions

- What is Bayes' theorem, and how is it used for classification?
- Why is the algorithm called "naïve"?
- What is the difference between prior, likelihood, and posterior probability?
- Why is the Gaussian distribution used to model continuous features in GaussianNB?
- What is Laplace (additive) smoothing, and why is it needed?
- How does Naïve Bayes handle features that are correlated?
- Why can Naïve Bayes perform well even when its independence assumption is unrealistic?
- What is the effect of class imbalance on the prior probability?
- How does feature scaling affect Gaussian Naïve Bayes compared to distance-based classifiers?
- What are the main advantages and limitations of Naïve Bayes classifiers?

Expected Deliverables

Students should submit:

- Python notebook containing the complete experiment.
- Dataset exploration and probability calculations.
- Implementation of Naïve Bayes from scratch for selected samples.
- Scikit-Learn model results.
- Confusion matrix and classification metrics.
- A short analysis/conclusion (150–200 words) explaining the strengths and limitations of Naïve Bayes for this dataset.